

# Judge0 API 연동 공유 문서

CodeYar Backend / backend 브랜치 / 팀 공유용

이 문서는 Judge0 담당 구현 범위와 프론트/백 팀원이 실제 API를 호출할 때 보내야 할 값, 받을 값, 설정값, 테스트 흐름을 한 곳에 정리한 문서입니다.

## 1. 현재 Judge0 담당 구현 상태

- Judge0 batch 제출, Base64 인코딩, 백그라운드 polling 흐름이 구현되어 있습니다.
- 학생 제출은 problem\_id와 code만 받고, 채점 조건은 DB의 Problem에서 읽습니다.
- AST 검사를 통과한 제출만 Judge0로 전달됩니다.
- Problem.memory\_limit은 MB로 저장하고 Judge0 요청 시 KB로 변환합니다.
- 결과 조회 응답에 total\_count, passed\_count, score가 포함됩니다.
- Judge0 연결 확인용 health check API가 추가되어 있습니다.

## 2. 전체 제출 흐름

- 1 학생이 로그인해서 access\_token을 받습니다.
- 2 프론트/클라이언트가 POST /api/v1/submissions/로 problem\_id와 code를 보냅니다.
- 3 백엔드가 Problem에서 language\_id, time\_limit, memory\_limit, test\_cases, ast\_conditions를 읽습니다.
- 4 AST 검사를 통과하면 Judge0 batch API에 테스트케이스별 제출을 생성합니다.
- 5 BackgroundTasks가 Judge0 결과를 polling하고 Submission DB를 업데이트합니다.
- 6 프론트/클라이언트가 GET /api/v1/submissions/{submission\_id}로 결과를 polling합니다.

## 3. API 빠른 참조

| API                                     | 누가 호출     | 보낼 것                                           | 받을 것                                     |
|-----------------------------------------|-----------|------------------------------------------------|------------------------------------------|
| POST /api/v1/submissions/               | 학생 클라이언트  | Authorization: Bearer , body: problem_id, code | 202, submission_id, status               |
| GET /api/v1/submissions/{submission_id} | 학생 클라이언트  | Authorization: Bearer                          | status, passed, score, results 등         |
| GET /api/v1/submissions/judge0/health   | 프론트/백 개발자 | 없음                                             | Judge0 연결 상태, mode, api_url, status_code |
| POST /api/v1/auth/login                 | 학생/교수     | username, password                             | access_token, token_type                 |

## 4. 코드 제출 API

### 4.1 요청

POST /api/v1/submissions/  
Authorization: Bearer <student\_token>  
Content-Type: application/json

```
{  
  "problem_id": "<problem_id>",  
  "code": "import sys\nprint(sum(map(int, sys.stdin.read().split())))"  
}
```

| 필드         | 타입     | 필수 | 설명                             |
|------------|--------|----|--------------------------------|
| problem_id | string | Y  | 제출할 문제 ID. Problem 테이블에 존재해야 함 |
| code       | string | Y  | 학생이 작성한 소스 코드                  |

주의: language\_id, time\_limit, memory\_limit, test\_cases, ast\_conditions는 제출 요청에 넣지 않습니다. 이 값들은 DB의 Problem에서 읽습니다.

## 4.2 성공 응답

HTTP 202 Accepted

```
{  
  "submission_id": "4d7f...",  
  "status": "Pending",  
  "message": null  
}
```

## 4.3 AST 실패 응답

HTTP 202 Accepted

```
{  
  "submission_id": "4d7f...",  
  "status": "AST Fail",  
  "message": "반복문 사용이 금지되어 있습니다"  
}
```

## 4.4 주요 실패 케이스

| 상황                        | HTTP | detail                         |
|---------------------------|------|--------------------------------|
| problem_id가 DB에 없음        | 404  | 존재하지 않는 문제 ID입니다.              |
| Problem.test_cases가 비어 있음 | 400  | 이 문제에 등록된 테스트케이스가 없습니다.        |
| Problem의 JSON 필드 형식 오류    | 400  | test_cases JSON 형식이 올바르지 않습니다. |
| Judge0 API 인증/네트워크 오류     | 400  | Judge0APIError 메시지             |

## 5. 결과 조회 API

### 5.1 요청

GET /api/v1/submissions/{submission\_id}  
Authorization: Bearer <student\_token>

### 5.2 응답

HTTP 200 OK

```
{
  "submission_id": "4d7f...",
  "user_id": "student-id",
  "exam_id": "exam-id",
  "problem_id": "problem-id",
  "status": "Completed",
  "passed": true,
  "error_reason": null,
  "total_count": 3,
  "passed_count": 3,
  "score": 100,
  "results": [
    {
      "status_id": 3,
      "status_desc": "Accepted",
      "time": 0.01,
      "memory": 2048,
      "stdout": "3\n",
      "stderr": ""
    }
  ],
  "submitted_at": "2026-05-22T14:00:00"
}
```

| 필드           | 설명                                               |
|--------------|--------------------------------------------------|
| status       | Pending / Completed / Timeout / Error / AST Fail |
| passed       | 전체 테스트 통과 여부. 채점 중이면 null                        |
| error_reason | 실패/오류 사유. 통과 시 null                              |
| results      | 테스트케이스별 Judge0 결과 배열                             |
| total_count  | 전체 테스트케이스 수                                      |
| passed_count | Accepted 테스트케이스 수                                |
| score        | MVP 기준 전체 통과 100점, 일부 실패 0점, 결과 없음 null          |

## 6. Judge0 Health Check API

GET /api/v1/submissions/judge0/health

```
{
  "ok": true,
  "mode": "rapidapi",
  "api_url": "https://judge0-ce.p.rapidapi.com",
}
```

```

    "status_code": 200,
    "message": null
}

```

프론트/백 통합 전에 Judge0 서버나 RapidAPI 연결 상태를 확인할 때 사용합니다.

## 7. Problem DB에서 Judge0가 사용하는 값

| Problem 필드            | Judge0 사용 방식           | 비고                                 |
|-----------------------|------------------------|------------------------------------|
| language_id           | payload.language_id    | Python 3 기본값 71                    |
| time_limit            | payload.cpu_time_limit | 초 단위                               |
| memory_limit          | payload.memory_limit   | DB는 MB, Judge0 요청은 KB              |
| test_cases            | stdin, expected_output | JSON 배열 [{stdin, expected_output}] |
| ast_conditions        | Judge0 전 AST 검사        | 실패 시 Judge0 호출 안 함                 |
| global_ast_conditions | 전역 AST 검사              | 문제별 조건과 함께 검사                      |

```

test_cases 예시
[
  {"stdin": "1 2\n", "expected_output": "3\n"},
  {"stdin": "100 200\n", "expected_output": "300\n"}
]

```

## 8. 핵심 함수 및 클래스

| 위치                        | 이름                                | 역할                                           |
|---------------------------|-----------------------------------|----------------------------------------------|
| app/services/judge.py     | Judge0Client.submit_batch(...)    | Judge0 batch 제출. memory_limit MB -> KB 변환 포함 |
| app/services/judge.py     | Judge0Client.poll_and_update(...) | tokens polling 후 Submission DB 업데이트          |
| app/services/judge.py     | Judge0Client.health_check()       | /languages 호출로 연결 상태 확인                      |
| app/services/judge.py     | calculate_score(results)          | 전체 통과 100점, 일부 실패 0점                         |
| app/api/v1/submissions.py | submit_code(...)                  | 학생 제출 API 엔드포인트                              |
| app/api/v1/submissions.py | get_submission(...)               | 학생 결과 조회 API 엔드포인트                           |

## 9. Judge0 설정

| 환경변수                 | 예시                               | 설명                                     |
|----------------------|----------------------------------|----------------------------------------|
| JUDGE0_API_URL       | https://judge0-ce.p.rapidapi.com | Judge0 API base URL                    |
| JUDGE0_API_KEY       |                                  | RapidAPI key 또는 self-hosted auth token |
| JUDGE0_RAPIDAPI_HOST | judge0-ce.p.rapidapi.com         | RapidAPI Host header                   |
| JUDGE0_USE_RAPIDAPI  | true / false                     | true면 RapidAPI, false면 self-hosted     |

```
RapidAPI MVP
JUDGE0_API_URL=https://judge0-ce.p.rapidapi.com
JUDGE0_API_KEY=<rapidapi-key>
JUDGE0_RAPIDAPI_HOST=judge0-ce.p.rapidapi.com
JUDGE0_USE_RAPIDAPI=true
```

```
Self-hosted Judge0
JUDGE0_API_URL=http://localhost:2358
JUDGE0_API_KEY=
JUDGE0_USE_RAPIDAPI=false
```

## 10. 테스트 스크립트

Backend/test\_request.py는 학교 생성, 회원가입, 로그인, 테스트 문제 seed, 학생 제출, 결과 polling을 수행합니다.

```
cd Backend
python test_request.py

# DB 담당자가 문제 생성 API를 완성한 뒤 실제 문제로 테스트
CODEYAR_PROBLEM_ID=<problem-id> python test_request.py
```

## 11. 팀원별 연결 지점

| 담당              | 해야 할 일                                     | Judge0와 연결되는 부분                             |
|-----------------|--------------------------------------------|---------------------------------------------|
| DB/Auth         | 시험/문제 생성 API, 권한 검사, 결과 조회 API             | Problem 레코드가 있어야 실제 제출 가능                   |
| AST             | 문제 생성 시 ast_conditions 검증                  | AST 통과 시에만 Judge0 호출                        |
| Frontend/Client | 로그인, 문제 조회, 제출, 결과 polling UI              | 요청은 problem_id + code, 결과는 score/results 표시 |
| Judge0          | 채점 제출, polling, memory_limit, health check | 현재 정리 완료                                    |

## 12. 검증 명령

```
cd Backend
python3 -m unittest tests.test_judge tests.test_ast_checker
python3 -m compileall app test_request.py tests
```

- Judge0 memory\_limit MB -> KB 변환 테스트 통과.
- MVP 점수 계산 테스트 통과.
- 기존 AST checker 테스트 회귀 없음.

- 실제 Judge0 네트워크 E2E는 서버/API key 또는 self-hosted Judge0 실행 환경에서 test\_request.py로 확인합니다.